FoodBlock — Board Q&A

Q1: Blocks are permanent and public — so how do you revoke access to a secret like Coca-Cola's recipe when an employee leaves?

Blocks are immutable by design. Once written, they can't be deleted or changed. But some data — a recipe, a supplier's cost price, a farmer's private yield — can't be readable forever by anyone who once had access.

Answer

FoodBlock never stores the secret in plaintext. It stores encrypted ciphertext. Access is controlled by controlling the key — not the block.

Two layers:

Layer 1 — The block layer (permanent) Every access event is a block. `observe.access_grant` and `observe.access_revoke` are permanent audit records. Anyone can verify who had access and when — forever.

Layer 2 — The crypto layer (server-side, rotatable) The content is encrypted with a **Content Key (CK)**. The CK is wrapped in a **Master Key (MK)** that the operator holds. The block only contains ciphertext. To read it, you request the CK from the key server — which checks Layer 1 first.

The two-key trick: The Content Key never changes — so old blocks never need re-encryption. But the CK is wrapped in the MK. Revoke someone → rotate the MK → they can no longer unwrap the CK → every version of the chain, past and future, is instantly unreadable to them. One operation.

The Coca-Cola scenario:

1. Recipe encrypted with CK → ciphertext stored in block → block hash is public, content is not
2. Employee gets access → `observe.access_grant` block written
3. Employee leaves → `observe.access_revoke` block written → MK rotated
4. Ex-employee requests CK → key server checks Layer 1 → sees revocation → refuses

```
ingredients: "<ciphertext>"
```

The block is there. The key is not.

vs a database permission toggle:

	Traditional database	FoodBlock
Audit trail	Mutable logs	Permanent blocks
Revocation	Toggle a flag	Rotate the MK — cryptographically enforced
Re-encryption on revoke	Required	Not required
Operator sovereignty	Vendor holds data	Coca-Cola holds their own key store

On short-lived tokens: The current model has one gap — a leaked CK is valid forever. The fix is small: the key server returns a short-lived token (valid hours, not forever) instead of the raw CK. Damage window goes

from forever to hours. Same security posture as AWS IAM, Google, and Apple. Defence in depth — not one perfect lock, but layers where each limits the blast radius of the one above it failing.

> ***Blocks prove what happened. Crypto controls who can read it.***

Q2: A warehouse camera captures a frame every second — wouldn't that make FoodBlock unusable at scale?

One camera at 1 frame/second = 86,400 blocks per day. A warehouse with 50 cameras = 4.3 million blocks per day. Most recording nothing — an empty aisle, an unchanged temperature. Every block must be hashed, signed, written, indexed, and trigger a database event. PostgreSQL is not a time-series database.

Answer

FoodBlock should not store raw telemetry. It should store proofs of telemetry.

Raw stream → time-series store. Camera feed goes to a fast database (InfluxDB, TimescaleDB). Fast writes, cheap storage, queryable. FoodBlock doesn't touch this layer.

Merkle anchoring — one block per interval.

Think of a tournament bracket. 8 teams play → 4 results → 2 results → 1 champion. The champion represents the entire tournament. If any game went differently, you'd have a different champion. One name proves everything.

Merkle root applies the same idea to sensor data:

```
Reading 1: 4.1°C  ─┐
Reading 2: 4.1°C  ─┴─ hash AB ─┐
Reading 3: 4.2°C  ─┐            ├─ ROOT (one block in FoodBlock)
Reading 4: 4.1°C  ─┴─ hash CD ─┘
```

One anchor block per hour instead of 86,400. Any individual reading is verifiable against the root on demand. If anyone disputes a reading, pull it from the fast store and prove it matches the root. Cannot be faked. Bitcoin has run this mechanism at global scale since 2009.

The better answer: the camera as an agent.

A lightweight AI model runs on the device. It watches the feed and compresses it — not to frames, but to meaning.

```
86,400 frames/day
→ "Empty aisle, no anomalies"     →  discarded
→ "Person in restricted zone 14:32"  →  block
→ "Temperature threshold crossed 14:47"  →  block
```

3–4 blocks per day. Raw footage never leaves the device. Regulators get cryptographic proof. Nobody gets the footage. The camera is an `actor.agent` block with its own Ed25519 keypair — you know not just what was recorded but which physical device recorded it.

> ***FoodBlock captures proofs, not data. The raw data lives in a fast store. Proofs live in FoodBlock.***

Q3: Storing millions of blocks sounds expensive — isn't this the same problem as blockchain?

If every product, order, review, and sensor event becomes a block across the food industry, doesn't the cost spiral like Ethereum gas fees?

Answer

FoodBlock is not a blockchain. The name is misleading.

Blockchain is expensive because of **consensus** — every node in the network must agree on every transaction. Bitcoin does this through proof of work (mining). Ethereum uses validators. Writing one transaction to Ethereum costs gas — $5–50+ per write during congestion. The cost is the agreement process, not the storage.

FoodBlock has none of that. A FoodBlock write is a database insert into PostgreSQL. No mining. No validators. No consensus. No gas.

The actual numbers (1 block ≈ 1KB):

Scale	Storage	Hot cost/month	Cold cost/month
1 million blocks	~1 GB	~$0.10	~$0.02
10 million blocks	~10 GB	~$1	~$0.23
100 million blocks	~100 GB	~$10	~$2.30
1 billion blocks	~1 TB	~$100	~$23

Hot = AWS RDS PostgreSQL. Cold = AWS S3.

For comparison: 1 million Ethereum transactions at ~$1 average gas = $1,000,000. 1 million FoodBlocks = $0.10 in storage.

At scale, recent data stays hot and older history migrates to cold. The hash is the address — any block is retrievable from cold storage on demand. The hot bill stays flat regardless of total history size.

> ***The economics get better at scale, not worse.***

Q4: With thousands of actors writing blocks, how do you stop the same thing being recorded twice — or 100 identical bags collapsing into one block?

Three distinct duplicate problems exist: accidental technical duplicates (network retries, double submits), semantic duplicates (10,000 farmers all describing wheat), and physical instance duplicates (100 identical bags that are actually separate objects).

Accidental duplicates — already solved

If type, state, and refs are byte-for-byte identical, the SHA-256 hash is identical, and the database PRIMARY KEY rejects it. Exact duplicates cannot exist by design.

Semantic duplicates — refs solve it

10,000 farmers describing wheat aren't duplicates — they're 10,000 different products from 10,000 different farms. Different authors, different harvests, different hashes. The protocol sees them correctly as distinct. Vocabulary blocks provide shared field naming so they're comparable, not merged.

Physical instances — lot blocks + mini blocks

A mill produces 100 identical bags. Same content → same hash → content-addressing collapses them to 1 block. The other 99 have no identity.

The solution:

1 — Lot block. One block for the production run. All upstream provenance lives here — farm, harvest, certifications. All 100 bags share this.

2 — Mini blocks. A vision camera creates a lightweight block for each bag as it comes off the line. Just a position number, a timestamp, and a ref to the lot. Each bag gets its own hash, its own identity, its own exact production timestamp.

```
substance.product_instance {
  state: { position: 23, produced_at: "09:03:45" }
  refs: { lot: lot_block_hash }
}
```

100 mini blocks costs fractions of a cent. Each inherits full provenance by following refs.lot.

3 — Transfer chain. When a merchant buys bags, a `transfer.order` block references the mini blocks and the buyer. The full chain:

```
Farm → Lot block → Mini block (bag 23) → transfer.order → Merchant
```

Backward: trace any bag to its farm. Forward: trace any bag to who bought it.

4 — Defect on bag 23. Mini block already exists. Camera detected it at 09:03:45. `observe.defect` chains to it instantly. Temperature spike between 10:15–10:22 → query all mini blocks with `produced_at` in that window → instant recall list.

5 — QR / barcode. The mini block hash IS the physical identity. Printed as a QR at the moment the camera creates the block. Physical label and digital record born simultaneously. Scan the bag → resolve to the block → trace the full chain. If the product already has a GS1 barcode, it goes into the mini block state as a field.

	Old model	FoodBlock
Individual item identity	External barcode system	Mini block hash IS the identity
Production timestamp	Log file	In the mini block
Provenance	Repeated per item	Inherited via refs.lot — stored once
Defect tracing	Manual, lot-level	observe.defect on the exact mini block
Field to shelf	Multiple disconnected systems	One chain, both directions

---

Q5: The food industry has FDA labels, GS1 databases, and SAP systems — none built for FoodBlock. How does legacy data get in without being re-entered from scratch?

Regulatory formats are legally fixed — FDA and EU mandate specific fields. GS1 has hundreds of millions of product records. Manufacturers run ERP systems with years of data. None of it speaks FoodBlock. And if FoodBlock owns all the mappings, it breaks every time a regulation changes.

Answer

FoodBlock does not replace legacy formats. It wraps them.

FoodBlock is the envelope, not the format. Whatever legacy data needs to exist — FDA nutrition label, GS1 product record, EU allergen declaration — goes into block state verbatim. The regulatory format is preserved exactly as required by law. FoodBlock adds the hash, chain, and provenance on top without touching the content.

The analogy is a shipping container. The container doesn't care what's inside. It provides the standard interface. The goods are whatever they need to be.

Separate blocks, not embedded. The product block is the stable anchor — name, GTIN, manufacturer. Everything else attaches to it as separate blocks.

```
substance.product { name: "Organic Whole Milk", gtin: "0614141000036" }
    ↑
observe.nutrition_label    (FDA format, 2020 formula)
observe.nutrition_label    (FDA format, 2024 reformulation)
observe.nutrition_label    (EU format)
observe.certification      (organic cert)
observe.allergen_declaration
```

Nutrition data changes independently from product identity. A reformulation creates a new nutrition label block — the product block stays untouched. Different markets get different label blocks on the same product. Different parties (manufacturer, cert body, regulator) each attach their own blocks without touching each other's.

Existing identifiers go in state. GS1 GTIN, FDA facility numbers, EU operator numbers go directly into block state. FoodBlock extends the existing identifier system — it doesn't compete with it.

Vocabulary blocks handle schema evolution. When FDA updates its label requirements, a new vocabulary block is created (chain update). Old data still references the old vocabulary — still valid. New data references the new version. FoodX does not own these vocabularies. The community does. A food standards body publishes the FDA vocabulary. FoodX owns the envelope. The community owns the mappings.

AI is the translation layer. The `fb()` entry point parses natural language into blocks. The same mechanism handles structured legacy formats — GS1 feeds, SAP exports, nutrition label XML. Manufacturers export from their existing system. AI translates it into blocks. Their workflow barely changes.

> ***FoodBlock owns the envelope. Operators own the contents. The community owns the mappings.***

Q6: If FoodBlocks are public, can anyone see a restaurant's order prices and margins?

Answer

Not all blocks are public. Every block has a visibility level set at creation.

Visibility	Who can read it	Used for
`public`	Everyone	Products, reviews, certifications, profiles
`network`	Only parties in the transaction	Orders, shipments, offers
`direct`	Only named recipients	Messages, payments, subscriptions

A restaurant's order is `visibility: network` — only the restaurant and supplier can read it. Payments and conversations are `visibility: direct`. Financial data never surfaces publicly.

The provenance that is public — a product's origin, a farm's certification, a food safety record — is public by design. That is the trust layer the system is built on. The commercial relationship on top of it is private by design.

> ***The protocol is public. The transactions are not.***
